UNIVERSITÉ LOYOLA DU CONGO

Faculté des Sciences et Technologies

Génie Informatique

MON MIAM MIAM

CONVENTIONS COMMUNES DE L'API

Spécifications techniques communes

Projet d'application web pour ZeDuc@Space

Nature du document : Référentiel technique commun

Périmètre : API et intégration des modules

Statut : Proposition à valider par l'équipe

Année académique 2026–2027

1. Introduction

Mon Miam Miam est une application web destinée à faciliter les commandes de repas auprès du restaurant ZeDuc@Space.

La réalisation de cette application nécessite plusieurs modules, notamment la gestion des utilisateurs, des commandes, des paiements, de la fidélité, du parrainage et des promotions.

Pour permettre à ces modules de communiquer correctement, l'équipe doit adopter des conventions techniques communes.

Ce document définit des règles proposées concernant le format des réponses, les codes HTTP, les erreurs, le nommage des routes, les statuts métier et les données échangées.

Ces conventions constituent une base de travail qui devra être validée collectivement avant l'implémentation des API.

Objectif du document

Harmoniser les échanges entre les modules de Mon Miam Miam afin de faciliter leur intégration, leur maintenance et leur évolution.

2. Principes généraux

Les API du projet devront respecter les principes suivants :

- **Cohérence :** tous les modules suivent les mêmes conventions de communication.
- **Lisibilité :** les noms des routes, des champs et des erreurs doivent être explicites.
- **Sécurité :** les données sensibles doivent être protégées et les accès contrôlés.
- **Validation :** les données reçues doivent être vérifiées avant tout traitement.
- **Traçabilité :** les opérations importantes doivent pouvoir être suivies à l'aide de journaux appropriés.
- **Maintenabilité :** les conventions doivent être documentées pour faciliter le travail de chaque développeur.

3. Format des échanges

3.1. Format des données

Le format JSON (*JavaScript Object Notation*) est proposé comme format commun pour les échanges entre le client et les API.

Les requêtes contenant un corps JSON devront utiliser l'en-tête :

```
Content-Type: application/json
```

Les réponses JSON devront, lorsque cela est applicable, utiliser :

```
Content-Type: application/json
```

Les noms des champs seront écrits en `snake_case`, c'est-à-dire en minuscules avec des mots séparés par des traits de soulignement.

Exemples : `user_id`, `points_balance`, `referral_code`, `created_at`.

3.2. Structure standard d'une réponse réussie

Toutes les API devront suivre une structure cohérente. La structure ci-dessous est une proposition commune :

```
{
  "success": true,
  "message": "Operation reussie",
  "data": {
    "user_id": 15,
    "points_balance": 120
  },
  "error": null
}
```

Les champs ont les significations suivantes :

Champ	Signification
`success`	Indique si l'opération a réussi.
`message`	Fournit un message lisible.
`data`	Contient le résultat de l'opération.
`error`	Contient les informations d'erreur ou `null`.

TABLE 1 – Structure proposée d'une réponse API

Pour une opération réussie, `success` vaut `true` et `error` vaut `null`. Le champ `data` peut être un objet, une liste ou `null`, selon l'opération.

3.3. Structure standard d'une réponse en erreur

Lorsqu'une requête échoue, la réponse devra respecter la même structure générale.

```
{
  "success": false,
  "message": "Utilisateur introuvable",
  "data": null,
  "error": {
    "code": "USER_NOT_FOUND",
    "details": null
  }
```

```
}
```

Le champ `code` représente un identifiant stable de l'erreur. Le champ `details` peut fournir des informations de validation supplémentaires, sans exposer de données sensibles.

Le code HTTP devra également correspondre au type d'erreur rencontré.

4. Codes HTTP

Les codes HTTP permettent d'indiquer le résultat d'une requête.

Code	Signification	Exemple d'utilisation
200	OK	Consultation réussie du solde de points.
201	Created	Création réussie d'un code de parrainage.
204	No Content	Suppression réussie sans contenu à renvoyer.
400	Bad Request	Requête mal formée ou paramètres invalides.
401	Unauthorized	Authentification absente ou invalide.
403	Forbidden	Utilisateur authentifié mais non autorisé.
404	Not Found	Ressource ou code de parrainage introuvable.
409	Conflict	Opération incompatible avec l'état actuel.
422	Unprocessable Content	Données reçues mais refusées par les règles métier.
429	Too Many Requests	Trop de requêtes dans un intervalle donné.
500	Internal Server Error	Erreur interne inattendue du serveur.

Table 2: Codes HTTP proposés pour le projet

Les développeurs devront choisir le code le plus adapté à chaque situation et conserver la même logique dans tous les modules.

Par exemple, un solde de points insuffisant peut être signalé par `422`, tandis qu'un utilisateur qui tente d'accéder à une fonction réservée à l'administration peut recevoir `403`.

5. Conventions de gestion des erreurs

Les erreurs doivent être identifiables et compréhensibles afin de faciliter leur traitement par le client et leur diagnostic par l'équipe.

Code d'erreur	HTTP proposé	Signification
VALIDATION_ERROR	422	Une ou plusieurs données ne respectent pas les règles.
UNAUTHORIZED	401	Authentification absente ou invalide.
FORBIDDEN	403	Droits insuffisants pour l'opération.
RESOURCE_NOT_FOUND	404	Ressource introuvable.
INSUFFICIENT_POINTS	422	Solde de points insuffisant.
REFERRAL_CODE_INVALID	422	Code de parrainage invalide ou inutilisable.
REFERRAL_CODE_ALREADY_USED	409	Code déjà utilisé dans un contexte où il ne peut pas l'être à nouveau.
PROMOTION_INACTIVE	422	Promotion inactive ou hors période de validité.
RATE_LIMIT_EXCEEDED	429	Limite de requêtes atteinte.
INTERNAL_ERROR	500	Erreur inattendue du serveur.

Table 3 – Exemples de codes d'erreur communs

Ces identifiants sont proposés à titre de convention. L'équipe devra les compléter selon les besoins réels des différents modules.

Les messages destinés aux utilisateurs ne devront pas contenir de mots de passe, de jetons d'authentification, de requêtes SQL ou de détails techniques internes.

6. Conventions de nommage des routes

6.1. Principes

Les routes devront respecter les règles suivantes :

— Utiliser une base commune, par exemple `/api/v1`.
— Utiliser des noms de ressources au pluriel.
— Utiliser les méthodes HTTP selon l'opération attendue.
— Employer des noms explicites et cohérents.
— Protéger les routes selon les rôles et les droits requis.

6.2. Méthodes HTTP

6.3. Exemples de routes proposées

Les routes suivantes sont des exemples de conception. Elles ne signifient pas que les API sont déjà développées.

Méthode	Usage	Exemple
GET	Lire une ressource	Consulter le solde de points.
POST	Créer ou déclencher une opération	Utiliser un code de parrainage.
PUT	Remplacer une ressource	Remplacer une configuration complète.
PATCH	Modifier partiellement	Modifier une promotion.
DELETE	Supprimer une ressource	Supprimer une promotion si autorisé.

Table 4 – Méthodes HTTP et usages

Méthode	Route proposée	Fonction
GET	`/api/v1/loyalty/balance`	Consulter le solde de points.
GET	`/api/v1/loyalty/history`	Consulter l'historique des points.
POST	`/api/v1/referrals`	Créer un code de parrainage.
POST	`/api/v1/referrals/use`	Utiliser un code de parrainage.
GET	`/api/v1/referrals`	Consulter les parrainages de l'utilisateur.
GET	`/api/v1/promotions`	Consulter les promotions disponibles.
POST	`/api/v1/admin/promotions`	Créer une promotion côté administration.
PATCH	`/api/v1/admin/promotions/{id}`	Modifier une promotion.
GET	`/api/v1/leaderboard`	Consulter le classement.
GET	`/api/v1/admin/statistics`	Consulter les statistiques administratives.

Table 5: Exemples de routes par module

Les noms exacts devront être harmonisés avec les responsables des modules utilisateurs, commandes, paiements et administration.

7. Conventions relatives aux statuts métier

Les statuts indiquent l'état actuel d'une ressource. Ils doivent être définis clairement et utilisés de manière cohérente.

7.1. Parrainage

Les statuts proposés pour le suivi d'un parrainage sont :

Statut	Signification
PENDING	Parrainage créé, mais récompense non encore attribuée.
COMPLETED	Conditions de réussite du parrainage remplies.
REWARDED	Récompense attribuée après validation des conditions.
CANCELLED	Parrainage annulé selon les règles définies.

TABLE 6 – Statuts proposés pour le parrainage

Selon les règles métier du projet, la récompense du parrain est attribuée après la première commande réussie du filleul. La transition vers l'état `REWARDED` ne doit intervenir qu'après vérification de cette condition et confirmation de l'attribution effective.

7.2. Promotion

Les statuts proposés pour les promotions sont :

- `DRAFT` : promotion en préparation ;
- `SCHEDULED` : promotion programmée ;
- `ACTIVE` : promotion actuellement applicable ;
- `EXPIRED` : période de validité terminée ;
- `CANCELLED` : promotion annulée.

Le statut effectif devra être cohérent avec les dates de validité et les règles définies par l'administration.

7.3. Historique des points

Les opérations de fidélité devront permettre de distinguer les différents mouvements de points. Les types proposés sont :

- `EARNED` : points gagnés ;
- `REDEEMED` : points utilisés ;
- `REFERRAL_REWARD` : points attribués pour un parrainage ;
- `EXPIRED` : points arrivés à expiration, si cette règle est retenue ;
- `ADJUSTED` : correction administrative justifiée.

Chaque mouvement devrait conserver sa date, son type, sa quantité et une référence permettant de retrouver l'opération associée.

8. Conventions relatives aux données

8.1. Identifiants

Les identifiants des ressources devront suivre une convention commune. L'équipe devra choisir le type d'identifiant adapté à son modèle de données et l'utiliser uniformément.

Les routes utilisant un identifiant devront vérifier que la ressource existe et que l'utilisateur a le droit d'y accéder.

8.2. Dates et heures

Les dates et heures échangées par les API devront suivre le format ISO 8601, avec un fuseau horaire explicite.

Exemple :

```
{
  "created_at": "2026-10-26T14:30:00Z"
}
```

Le suffixe `Z` indique que l'heure est exprimée en UTC. Les interfaces pourront convertir cette heure vers le fuseau horaire local pour l'affichage.

8.3. Montants financiers

Les montants devront être associés à une devise explicite. Pour Mon Miam Miam, le franc congolais (CDF) pourra être utilisé lorsque cela correspond aux règles de paiement du restaurant.

Exemple :

```
{
  "amount": 10000,
  "currency": "CDF"
}
```

L'équipe devra décider d'une représentation cohérente des montants et éviter les conversions imprécises. La devise ne devra pas être supposée lorsque plusieurs devises sont possibles.

8.4. Points de fidélité

Le solde et les mouvements de points devront utiliser une représentation numérique cohérente. Si les points sont indivisibles, les valeurs seront des entiers positifs ou nuls pour les soldes.

Exemple :

```
{
  "user_id": 15,
  "points_balance": 120
}
```

Les règles de conversion entre dépenses et points, ainsi que la valeur des réductions, devront être définies dans les règles métier validées par l'équipe.

9. Authentification et autorisation

Les routes qui exposent des informations personnelles ou permettent de modifier des données devront être protégées.

Les conventions de sécurité proposées sont les suivantes :

— Vérifier l'identité de l'utilisateur pour les routes protégées.
— Contrôler les droits avant chaque opération sensible.
— Réserver les routes d'administration aux rôles autorisés.
— Ne jamais exposer les mots de passe ou les jetons secrets.
— Utiliser HTTPS en environnement de production.
— Valider les entrées et limiter les tentatives abusives.
— Journaliser les opérations administratives importantes.

L'équipe devra choisir une méthode d'authentification commune et documenter la façon dont les jetons ou sessions sont transmis.

Une route ne doit jamais accorder un accès administrateur simplement parce que le client transmet un champ indiquant un rôle.

10. Pagination et listes de résultats

Les routes renvoyant des listes, par exemple l'historique des points ou les utilisateurs parrainés, devront prévoir une pagination lorsque le nombre de résultats peut devenir important.

Une convention possible consiste à utiliser les paramètres `page` et `limit`.

Exemple de requête :

```
GET /api/v1/loyalty/history?page=1&limit=20
```

La réponse pourrait contenir :

```
{
  "success": true,
  "message": "Historique recupere",
  "data": {
```

```
    "items": [],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 0
    }
  },
  "error": null
}
```

Les limites maximales devront être définies pour éviter les requêtes trop volumineuses.

11. Validation des données

Toute donnée reçue par une API doit être validée avant son utilisation.

Exemples de vérifications :

— vérifier la présence des champs obligatoires ;
— contrôler le type et le format des valeurs ;
— vérifier l'existence des identifiants fournis ;
— refuser l'utilisation de points si le solde est insuffisant ;
— vérifier la période de validité d'une promotion ;
— empêcher les réutilisations interdites d'un code de parrainage ;
— vérifier les droits de l'utilisateur avant la modification d'une ressource.

Les validations côté interface améliorent l'expérience utilisateur, mais elles ne remplacent jamais les validations effectuées côté serveur.

12. Responsabilités de l'équipe

Pour assurer une intégration correcte, chaque responsable de module devra :

1. respecter les conventions validées par l'équipe ;
2. documenter les routes de son module ;
3. préciser les paramètres attendus et les réponses possibles ;
4. utiliser les codes HTTP et erreurs communs ;
5. signaler les besoins qui nécessitent une modification des conventions ;
6. vérifier la compatibilité de son module avec les autres ;
7. participer à la revue croisée avant l'intégration finale.

Toute modification des conventions communes devra être communiquée aux membres concernés afin d'éviter les incompatibilités.

13. Conclusion

Les conventions communes constituent une base importante pour l'intégration des différents modules de Mon Miam Miam.

L'utilisation d'un format JSON homogène, de codes HTTP appropriés, de routes cohérentes, de statuts documentés et de règles de sécurité communes permettra de simplifier le développement et la maintenance de l'application.

Les propositions de ce document devront être discutées et validées par les membres de l'équipe avant leur adoption définitive.

Une fois validées, elles serviront de référence lors de l'implémentation, des tests et de l'intégration des API.